

Code Documentation

Project overview

- Objective: Predict maintenance needs for a robotic welding arm by fusing sensor-based anomaly prediction (Random Forest, LSTM) with computer vision seam detection (YOLOv5).
- Deliverables:
 - CLI (main.py) for sensor-only, vision-only, and full pipeline runs.
 - Streamlit app (app.py) for operator-friendly UI.
 - Reproducible environment via UV (pyproject.toml).

System architecture

- Data inputs:
 - Sensor CSV with numeric features and optional label column (Defect).
 - Single image for seam detection.
- Models:
 - Random Forest (tabular).
 - LSTM (sequence modeling over sensor time steps).
 - YOLOv5 (object detection for seams).
- Outputs:
 - Sensor prediction: {0,1} and a float confidence.
 - Vision prediction: integer seam count (lines in YOLO label files).
 - Final verdict: “ Healthy” if sensor=0 and seams>0, else “ Maintenance Required”.

Data contracts

- Sensor CSV:
 - Columns: multiple numeric features; if Defect exists, it is dropped at inference.
 - Inference rule: last row used by default unless pipeline implements a selection.
- Image:
 - Formats: jpg, jpeg, png.
 - Single-file inference.

Directory layout

- app.py: Streamlit UI.
- main.py: CLI and orchestration.
- models/
 - weld_yolo/weights/best.pt (YOLOv5 weights).
 - rf.pkl, lstm.pth (example artifacts, if used).
- utils/:

- `paths.py`: `normalize_path` for Windows→WSL path conversion.
 - `init.py` (required to import `utils` as a package).
- `docs/`:
 - `vision_preds/`: annotated image outputs.
 - `vision_preds/labels/`: YOLO label text files.
- `data/`: local datasets.
- `notebooks/`: research and training notebooks.

Module: `utils/paths.py`

Purpose: Reliable path normalization across Windows/WSL/Linux.

Key function:

- `normalize_path(p: str) -> Path`
 - Input: raw path string (Windows or POSIX).
 - Behavior:
 - If Windows drive path like `C:\...`, convert to `/mnt/c/...` for WSL.
 - Otherwise resolve and expanduser.
 - Returns: `pathlib.Path` suitable for downstream I/O.

Module: `main.py`

Role: CLI entry point and programmatic API for sensor/vision/pipeline.

Globals and constants

- `MODELS_DIR`, `YOLOV5_DIR`, `YOLO_PROJECT_NAME`: resolve weights and detect.py locations.
- `DEVICE`: torch device selection (`cpu/cuda`).

Function: `predict_sensor(args) -> tuple[int, float]`

- Inputs:
 - `args.model`: “rf” or “lstm”.
 - `args.sensor_csv`: path to CSV (string).
 - `args.force_fail`: bool (if true, returns (1, 0.95) for testing).
- Process:
 - Read CSV into DataFrame.
 - Drop Defect column if present during inference.
 - For RF:
 - Load RF model artifact and feature order.
 - Extract last row; output (pred, confidence).
 - For LSTM:
 - Load LSTM weights.
 - Build/reshape a sequence tensor for the last N steps.
 - Output (pred, confidence).
- Outputs:
 - `pred`: int (0=OK, 1=Maintenance).

- confidence: float within [0,1].

Assumptions:

- Models are trained and artifacts exist at expected paths.
- LSTM sequence length handled by preprocessing.

Function: predict_vision(args) -> int

- Inputs:
 - args.image: path to image string.
 - args.conf: float (0–1), YOLO confidence threshold.
 - args.iou: float (0–1), YOLO IoU threshold.
 - args.clear_outputs: bool; if true, deletes docs/vision_preds before run.
- Process:
 - Normalize image path (normalize_path).
 - Invoke YOLOv5 detect.py with:
 - --weights models/weld_yolo/weights/best.pt
 - --img 640
 - --conf {args.conf}
 - --iou {args.iou}
 - --source {image_path}
 - --project {absolute docs path}
 - --name vision_preds
 - --exist-ok --save-txt --save-conf
 - Important: run subprocess with cwd=project root so outputs go to docs/vision_preds (not yolov5/docs).
 - After run, count seam detections by summing lines across docs/vision_preds/labels/*.txt.
- Output:
 - seams_detected: int.

Edge cases handled:

- Missing image or weights: returns 0.
- Missing labels directory: returns 0 and logs info.

Function: run_pipeline(args) -> dict

- Inputs:
 - args.model, args.sensor_csv, args.image, args.conf, args.iou, args.clear_outputs, args.force_fail.
- Process:
 - Calls predict_sensor then predict_vision.
 - Combines results into a verdict.
- Output:
 - Dict with fields: sensor_pred, sensor_conf, seams, verdict.

CLI commands

- `predict-sensor --model {rf|lstm} --sensor-csv PATH [--force-fail]`
- `predict-vision --image PATH [--conf 0.25] [--iou 0.45] [--clear-outputs]`
- `pipeline --model rf --sensor-csv PATH --image PATH [--conf --iou --clear-outputs] [--force-fail]`

Module: `app.py` (Streamlit UI)

Purpose: Operator-facing UI for running sensor-only, vision-only, or full pipeline.

Sidebar controls

- **Sensor model:** `rf/lstm`.
- **Uploads:** CSV (sensor), image (vision).
- **Vision thresholds:** confidence, IoU.
- **Flags:** clear outputs, force fail.

Helpers

- `_persist_upload_to_tmp(upload, suffix) -> Path`
 - Saves Streamlit upload to a temporary file for backend functions.
- `_find_latest_vision_outputs(project_root) -> (Path|None, Path|None)`
 - Returns annotated image and labels dir from `docs/vision_preds`.
- `_display_verdict(sensor_pred, sensor_conf, seams)`
 - Renders styled verdict card.
- `_display_csv_preview(upload)`
 - Shows `head(20)` and a line chart of up to 3 numeric columns.

Tabs and flows

- Full pipeline:
 - Requires both CSV + image.
 - Calls `predict_sensor` and `predict_vision` with explicit args.
 - Shows verdict card, annotated image, label summary.
- Sensor only:
 - Requires CSV.
 - Shows prediction and confidence.
 - Displays CSV preview and quick chart.
- Vision only:
 - Requires image.
 - Shows seams count, annotated image, label summary.

Output locations

- Annotated images and labels written to `docs/vision_preds/` and `docs/vision_preds/labels/`.

Models

Random Forest

- Training:
 - Standard scikit-learn pipeline (train/test split, StandardScaler optional).
 - Save model via joblib and persist feature order.
- Inference:
 - Read CSV, drop non-feature columns (Defect).
 - Use last row as input.
 - Probability from predict_proba for confidence.

LSTM

- Training:
 - PyTorch nn.LSTM or equivalent.
 - Sequence length N; targets reflect last time step.
 - Save state_dict.
- Inference:
 - Load state_dict.
 - Build tensor from last N rows; handle missing padding if needed.
 - Apply sigmoid/softmax for confidence.

YOLOv5

- Weights:
 - Place best.pt at models/weld_yolo/weights/best.pt.
- Inference:
 - detect.py invoked via subprocess with project/name args pointing to docs.
 - --save-txt --save-conf to write labels with confidences.

Final verdict logic

- Rule:
 - Healthy when sensor_pred == 0 AND seams > 0.
 - Maintenance Required otherwise.
- Rationale:
 - Sensor anomalies or missing seams both indicate risk.

Testing checklist

- Sensor only:
 - RF and LSTM both load and predict on the provided CSV.
 - Confidence returns in [0,1].
- Vision only:
 - YOLOv5 writes outputs to docs/vision_preds and labels to docs/vision_preds/labels.
 - Annotated image displays correctly in the app.
- Pipeline:
 - Combined verdict card matches the rule.
 - Force fail triggers sensor_pred=1.

- Cross-platform paths:
 - `normalize_path` correctly converts Windows paths under WSL.
- Thresholds:
 - Lower conf increases detections; higher IoU tightens matches.

Troubleshooting

- No labels created:
 - Ensure `detect.py` runs with `cwd=project root` (not `yolov5/`).
 - Verify flags: `--project docs --name vision_preds --exist-ok --save-txt --save-conf`.
- Streamlit deprecation:
 - Replace `use_column_width` with `use_container_width=True` for images.
- `ModuleNotFoundError: utils.paths`
 - Ensure `utils/init.py` exists and import as `from utils.paths import normalize_path`.
- LSTM shape errors:
 - Confirm sequence length used at inference matches training expectations.
- GPU issues:
 - Install CUDA-matched `torch/torchvision`, or force CPU by setting device to `cpu`.

Roadmap

- Session history panel in UI.
- Batch image processing and CSV queueing.
- Dockerfile + Streamlit Cloud deployment.
- Model monitoring (latency, drift).
- Dataset versioning and data cards.